

Software Engineering Assignment #1

AI Analysis

Access and Prompt Method of AI Used

For this assignment. I used **Gemini** and **ChatGPT**. Both models I accessed using their **website versions**. For Gemini, I used its **3.1 Pro model with extended thinking** for my prompt. And for ChatGPT, I toggled its **think mode**. Both models were accessed using a free account. The prompt I gave both AIs follows:

Write the code for a C program using the information below:

Overview

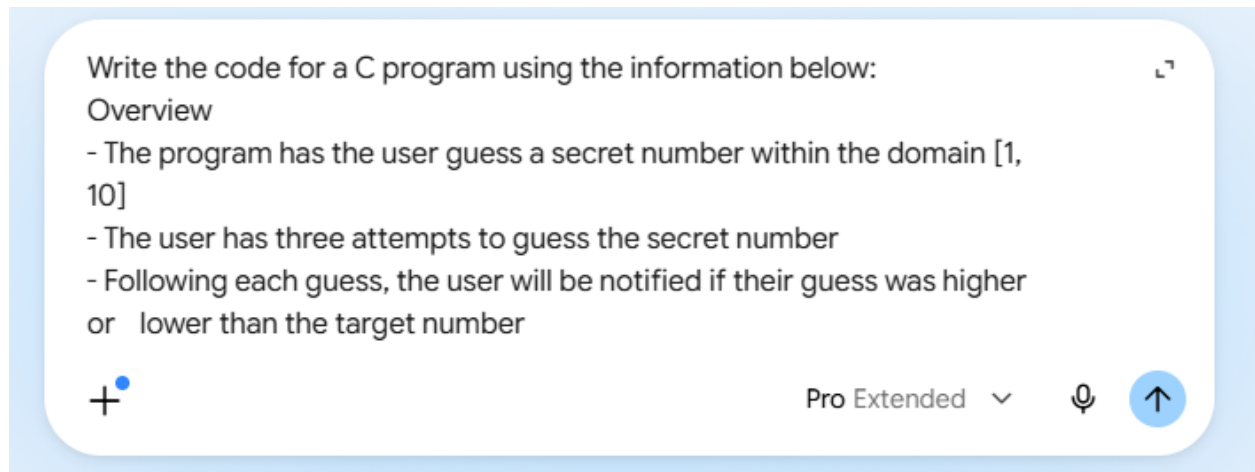
- The program has the user guess a secret number within the domain [1, 10]
- The user has three attempts to guess the secret number
- Following each guess, the user will be notified if their guess was higher or lower than the target number
- If the guess is correct or the number of attempts runs out, the game finishes

Technical

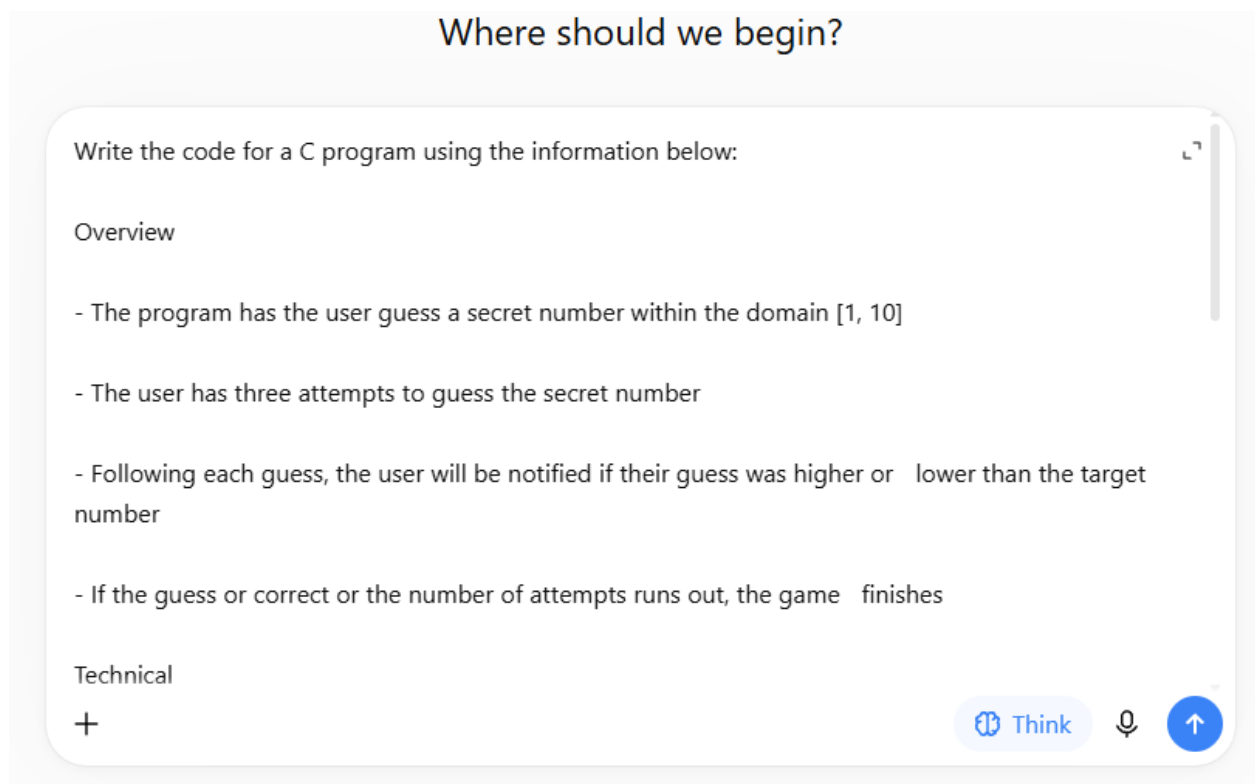
- The secret number is a constant value and not randomly assigned
- Input and output must be done with **printf** and **scanf**
- The three tries should be done in either a **for** or **while** loop
- Testing if a guess is high or low should be done in an **if-else** block
- Stop the loop if the player guesses the correct number

Output

- On start: "Guess a number between 1 and 10."
- On input: "Attempt (Attempt No.)/3. Enter your guess: "
- On guess low: "That guess is low."
- On guess high: "That guess is high."
- On guess equal: "That guess... is spot on!"
- On win: "You won the game!"
- On loss: "You lost the game. Better luck next time!"



(Fig. 1 – Gemini Setup)



(Fig 2. - ChatGPT Setup)

AI Analysis #1 – Gemini

```
1  #include <stdio.h>
2
3  int main() {
4      // The secret number is a constant value within the domain [1, 10]
5      const int SECRET_NUMBER = 7;
6      int guess;
7      int has_won = 0;
8
9      // On start output
10     printf("Guess a number between 1 and 10.\n");
11
12     // Loop for the three tries
13     for (int attempt = 1; attempt <= 3; attempt++) {
14         // On input output
15         printf("Attempt %d/3. Enter your guess: ", attempt);
16         scanf("%d", &guess);
17
18         // Testing if a guess is high, low, or correct using an if-else block
19         if (guess < SECRET_NUMBER) {
20             printf("That guess is low.\n");
21         } else if (guess > SECRET_NUMBER) {
22             printf("That guess is high.\n");
23         } else {
24             printf("That guess... is spot on!\n");
25             has_won = 1;
26             break; // Stop the loop if the player guesses the correct number
27         }
28     }
29
30     // Win/Loss condition outputs
31     if (has_won) {
32         printf("You won the game!\n");
33     } else {
34         printf("You lost the game. Better luck next time!\n");
35     }
36
37     return 0;
38 }
```

Correctness

Handles sample input but fails to handle unusual input. If something that is not an integer is given, then the program will not crash but has unusual behavior of printing attempt counts before giving the lose condition message.

Execution Time

The program has good execution time. A for loop is efficient and fits with the task. The if statements to compare the guess with the secret number starts with the more likely conditions that the guess is not equal to the secret number instead of checking equality first.

Space Complexity

The program currently uses four variables but could use only three. The four variables are “SECRET_NUMBER”, “guess”, “has_won”, and “attempt”. The guess and attempt variables are integral to the game and cannot be removed from the program without causing serious structural issues. SECRET_NUMBER could technically be removed and replaced with a magic number, but the damage to readability will be greater than the benefit for space saving since it is only a constant integer value. The only useless variable is has_won. After the for loop, the program determines if the user has won by using this variable. However, the condition for winning is that the guess variable matches the SECRET_NUMBER variable. Therefore, that check can be replaced with simply “guess == SECRET_NUMBER”.

Maintainability

The program has sparse comments that while helpful, lack depth into what each block of code is doing. Additionally, the program lacks proper error handling and checking. Besides these issues, the variable names are all descriptive, and the secret number variable has the correct format for a constant value.

AI Analysis #2 – ChatGPT

```
1  #include <stdio.h>
2
3  int main() {
4      const int secretNumber = 7;
5      int guess;
6      int won = 0;
7
8      printf("Guess a number between 1 and 10.\n");
9
10     for (int attempt = 1; attempt <= 3; attempt++) {
11         printf("Attempt %d/3. Enter your guess: ", attempt);
12         scanf("%d", &guess);
13
14         if (guess == secretNumber) {
15             printf("That guess... is spot on!\n");
16             won = 1;
17             break;
18         } else if (guess < secretNumber) {
19             printf("That guess is low.\n");
20         } else {
21             printf("That guess is high.\n");
22         }
23     }
24
25     if (won) {
26         printf("You won the game!\n");
27     } else {
28         printf("You lost the game. Better luck next time!\n");
29     }
30
31     return 0;
32 }
```

Correctness

Correctness is exactly the same as Gemini. Handles sample input but fails to handle unusual input. If something that is not an integer is given, then the program will not crash but has unusual behavior of printing attempt counts before giving the lose condition message.

Execution Time

The program has decent execution time. A for loop is efficient and fits with the task. However, the if statements first check if the guess is the secret number before checking the other conditions. It is more likely for the guess to not equal the secret number rather than for it to be exactly the secret number. Therefore, it would be generally more efficient if the greater than and lesser than checks were used with the equality condition triggering from the else.

Space Complexity

Its space complexity is exactly the same as Gemini's with the same improvement as well. Four variables can become three by removing the won check and replacing it with "guess == secretNumber".

Maintainability

The program is very unmaintainable. The program has zero comments whatsoever and no error checking. Furthermore, the constant variable secretNumber uses camel case which is very unusual for constant variables. And the variable name won does not adequately mark that it is used as a boolean value.

Choice of Template

I decided to base my code off of the Gemini output. Gemini produced code that was equal or superior to ChatGPT's code in all facets. Namely, the Gemini program has better time execution and maintainability than ChatGPT's code. Gemini's better condition checking and comments marked it as the better output between the two.

Final Program Improvements

Correctness

My final program checks if guess was able to correctly be assigned from the scanf. This makes my final program have more correctness than the Gemini program since it is able to handle error cases and unusual input without giving strange output or crashing.

Execution Time

My program ended up having slightly worse execution time than the Gemini program since I needed the extra checks for the scanned input being valid. However, the small increase in execution time is outweighed by the benefit to correctness that those checks provide.

Time Complexity

My program is less space complex than the Gemini program since it requires three variables as opposed to Gemini's four. This was done by replacing the has_won variable with a check at the end of the program that checked if "guess == SECRET_NUMBER".

Maintainability

My program is far more maintainable than the Gemini program because I included several comments that explained each line and what it was doing. This is much better than the sparse and general comments that Gemini's program had.